204252 Data Structures and Analysis

Chapter 8: Hashing (Part 1)

Thapanapong Rukkanchanunt and Jakarin Chawachat

August 17, 2026

Computer Science Department

Outline

General Hashing Concept

Hash Functions

Ideal Hashing Trace

General Hashing Concept

The Search Problem Revisited

So far, we have studied various data structures to store and retrieve elements:

- **Arrays / Lists**: Finding an element takes $\mathcal{O}(N)$ time (unsorted) or $\mathcal{O}(\log N)$ time (sorted array using binary search).
- **Binary Search Trees**: Average search time is $\mathcal{O}(\log N)$. Under worst-case unbalanced conditions, search degrades to $\mathcal{O}(N)$.
- **Balanced Search Trees (AVL, B-Trees)**: Guarantee $\mathcal{O}(\log N)$ search, insertion, and deletion time.

The Hashing Goal

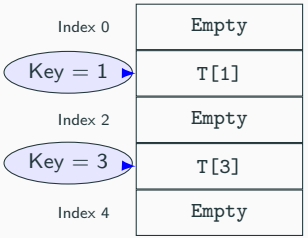
Can we do better? Can we perform searches, insertions, and deletions in **constant average time** $\mathcal{O}(1)$?

Direct Address Table (DAT)

Suppose keys are unique integers in a small range $[0, U-1]$. We can map keys directly to an array index.

Direct Addressing:

- Array T of size U.
- To insert key k, set $T[k] =$ element.
- Search is simply retrieving $T[k]$ in $\mathcal{O}(1)$ time.



Direct Address Table

Disadvantage

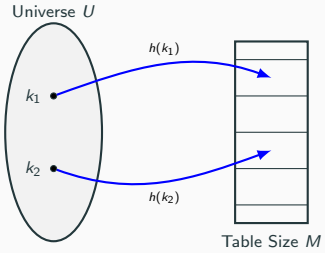
If the universe of possible keys U is extremely large (e.g. 9-digit student IDs, $U = 10^9$) but we only store $N = 100$ items,

The Hash Table Idea

To resolve the memory issues of direct addressing, we map a large universe of keys U to a much smaller table T of size TableSize ($M \ll U$) using a **Hash Function** h.

Definitions

- **Hash Function** h: A function that maps each key k to an index $i = h(k)$ in $[0, M-1]$.
- **Hash Value**: The integer $h(k)$ representing the slot where key k is stored.
- **Hash Table**: An array of size M that stores the elements.



Direct Addressing vs Hashing

Feature	Direct Address Table (DAT)	Hash Table
Table Size	Equal to size of universe U (extremely large)	Proportional to actual elements stored $M \approx N$
Space Complexity	$\mathcal{O}(U)$	$\mathcal{O}(M+N)$
Search Time	$\mathcal{O}(1)$ (Guaranteed)	$\mathcal{O}(1)$ average, $\mathcal{O}(N)$ worst-case
Collisions	Impossible (unique slot for each key)	Possible (multiple keys can map to same slot)
Key Type Support	Integers only	Any type (via custom hash functions)

Table 1: Comparison of Direct Addressing and Hashing

The Collision Concept

Since $M \ll U$, the Pigeonhole Principle guarantees that some keys will map to the same slot.

Collision Definition

A **Collision** occurs when two or more distinct keys $k_1 \neq k_2$ evaluate to the same hash value:

$$h(k_1) = h(k_2)$$

Example: $h(k) = k \bmod 10$

- $h(15) = 15 \bmod 10 = 5$
- $h(25) = 25 \bmod 10 = 5$
- Keys 15 and 25 collide at slot 5. They must be resolved!

Hash Functions

Hash Function Design Goals

An effective hash function should satisfy two primary requirements:

1. **Easy and Fast to Compute**: The computation must run in $\mathcal{O}(1)$ time relative to the table size.
2. **Uniform Distribution**: It should distribute keys evenly across all table cells to minimize collisions.

Why TableSize Should Be Prime

If `TableSize` is a prime number, it helps prevent clustering. If keys are not perfectly random (e.g., they share common factors like stepping in increments of 10), choosing a prime size ensures that keys spread out evenly across the table modulo space.

Hash Functions for Integers

For integer keys, the standard mapping strategy is:

$$h(k) = k \bmod \text{TableSize}$$

Bad Scenario: Even Table Size

Suppose TableSize $= 10$, and all input keys are multi-digit integers that end in 0 (e.g. $10, 20, 110, 250$).

- $h(10) = 0$, $h(20) = 0$, $h(110) = 0$, $h(250) = 0$.
- All keys map to slot 0, causing 100% collision rates and making search $\mathcal{O}(N)$.

Good Choice

Select a prime table size close to the expected load (e.g., TableSize $= 11$, which maps the keys to indices $10 \bmod 11 = 10$, $20 \bmod 11 = 9$, $110 \bmod 11 = 0$, $250 \bmod 11 = 8$).

Hash Functions for Strings: Method 1

Listing 1: Simple ASCII Sum

```
1 int hash( const string & key, int
      tableSize )
2 {
3     int hashVal = 0;
4     for( char ch : key )
5         hashVal += ch;
6     return hashVal % tableSize;
7 }
```

Analysis:

- Sums ASCII codes of characters.
- **Drawback**: Poor distribution.
- For example, if TableSize $= 10,007$ and keys have at most 8 chars (ASCII ≤ 127), the maximum sum is $127 \times 8 = 1016$.
- Slots $[1017, 10006]$ are never used!

Hash Functions for Strings: Method 2

Listing 2: Weighted First 3 Chars

```
1 int hash( const string & key, int
      tableSize )
2 {
3     return ( key[ 0 ] + 27 * key[ 1 ] +
4              729 * key[ 2 ] ) % tableSize;
5 }
```

Analysis:

- Assumes keys have ≥ 3 characters.
- 27 letters in alphabet + blank. $729 = 27^2$.
- **Drawback**: English is not random.
- Only $2,851$ of the $17,576$ three-letter combinations actually exist in English.
- Only 28% of the table will be hashed to, even under ideal conditions.

Hash Functions for Strings: Method 3

Listing 3: Polynomial rolling hash (Horner's Rule)

```
unsigned int hash( const string & key,
                   int tableSize )
{
    unsigned int hashVal = 0;
    for( char ch : key )
        hashVal = 37 * hashVal + ch;
    return hashVal % tableSize;
}
```

Analysis:

- Computes:

$$H = \sum_{i=0}^{L-1} \text{key}[L-1-i] \cdot 37^i$$

- Incorporates all characters.
- Uses **Horner's rule** to compute the polynomial efficiently:

$$((k_2 \cdot 37 + k_1) \cdot 37) + k_0$$

- Safe from overflow because it uses `unsigned int`.

Horner's Rule String Hashing Trace

Let's trace hashing the string `"cat"` using Horner's Rule with multiplier 37.

- ASCII codes: `'c'` = 99, `'a'` = 97, `'t'` = 116.

Step 1: Initialization

- String: `"cat"`
- Current character: None
- `hashVal` = 0

Horner's Rule String Hashing Trace

Let's trace hashing the string `"cat"` using Horner's Rule with multiplier 37.

- ASCII codes: `'c'` = 99, `'a'` = 97, `'t'` = 116.

Step 2: Process 'c'

- String: `"cat"` (ASCII = 99)
- `hashVal` $= 37 \times 0 + 99 = 99$

Horner's Rule String Hashing Trace

Let's trace hashing the string `"cat"` using Horner's Rule with multiplier 37.

- ASCII codes: `'c'` = 99, `'a'` = 97, `'t'` = 116.

Step 3: Process 'a'

- String: `"c`<u>`a`</u>`t"` (ASCII = 97)
- `hashVal` $= 37 \times 99 + 97 = 3663 + 97 = 3760$

Horner's Rule String Hashing Trace

Let's trace hashing the string `"cat"` using Horner's Rule with multiplier 37.

- ASCII codes: `'c'` = 99, `'a'` = 97, `'t'` = 116.

Step 4: Process 't'

- String: `"cat"` (ASCII = 116)
- `hashVal` $= 37 \times 3760 + 116 = 139120 + 116 = 139236$
- Final Hash Index = 139236 mod TableSize

Ideal Hashing Trace

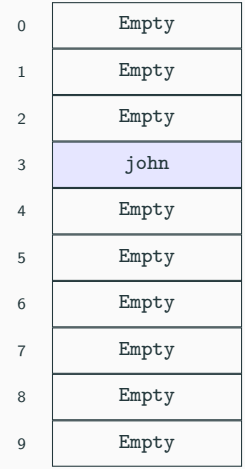
Ideal Hashing Trace: Setup

Let's trace inserting 4 records into a Hash Table of size 10 ($T[0 \ldots 9]$).

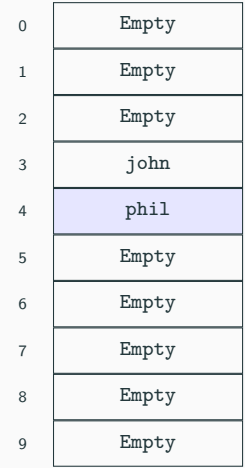
- Insertions: `"john"`, `"phil"`, `"dave"`, `"mary"`.
- Assume our ideal hash function maps:
  - $h(\text{"john"}) = 3$
  - $h(\text{"phil"}) = 4$
  - $h(\text{"dave"}) = 6$
  - $h(\text{"mary"}) = 7$

Ideal Hashing Trace (One slide per step)

Step 1: Insert "john" at index 3

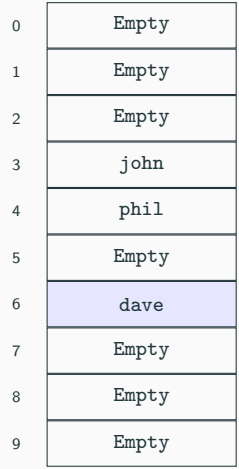


Ideal Hashing Trace (One slide per step)



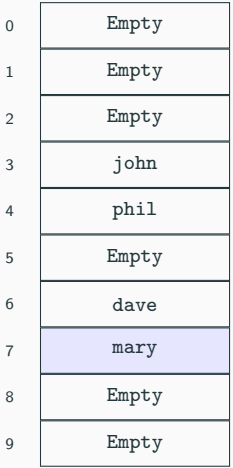
Step 2: Insert ”phil” at index 4

Ideal Hashing Trace (One slide per step)



Step 3: Insert ”dave” at index 6

Ideal Hashing Trace (One slide per step)



Step 4: Insert ”mary” at index 7